Learning Rust Through a Portable Graph Loader

A guided tour of the By the Bay scraper loader

Ownership, borrowing, data modeling, backend abstraction, and practical Rust design

Generated from the implementation in `src/bin/load_graph.rs`, `src/graph_loader.rs`, and `scripts/enrich_meetup_entities.py`

May 2026

Contents

Preface

This book teaches Rust through one concrete implementation: the By the Bay graph loader. The loader takes meetup and conference data, turns it into a portable graph model, and writes that graph to several database backends: FalkorDB, HelixDB, and SurrealDB.

The important part is not the databases. The important part is the Rust design: how data moves through the program, how ownership is preserved, where borrowing matters, how errors are propagated, how the command line maps to runtime configuration, and how backend-specific behavior is hidden behind a stable API.

The code discussed here lives in:

```
src/bin/load_graph.rs
src/graph_loader.rs
```

1. The Shape of the Program

The loader has two main layers.

The binary, `src/bin/load_graph.rs`, owns command-line parsing and input conversion. It reads either:

- a consolidated export, `--input-format export`
- Claude-style per-talk records, `--input-format talk-records`
- the enriched graph-record export at `data/enriched/bythebay-enriched-graph.json`, also through `--input-format talk-records`

Both input paths produce the same neutral value:

```
GraphData {
    nodes: Vec<GraphNode>,
    edges: Vec<GraphEdge>,
}
```

The library module, `src/graph_loader.rs`, owns database loading. Its public API is intentionally small:

```
pub fn load_graph(config: &GraphLoadConfig, graph: &GraphData) -> Result<()>
```

That tells you the whole contract:

- The caller owns the configuration.
- The caller owns the graph data.
- The loader borrows both.
- The loader either succeeds or returns an error.

This is a good Rust boundary. It is explicit, cheap to call, and difficult to misuse.

2. Command-Line Parsing With `clap`

The binary starts with a struct that describes the CLI:

```
#[derive(Debug, Parser)]
#[command(version, about = "Load By the Bay graph JSON into a graph database")]
struct Args {
    #[arg(short, long, default_value = "data/bythebay-graph.json")]
    input: PathBuf,

    #[arg(long, value_enum, default_value_t = InputFormat::Export)]
    input_format: InputFormat,

    #[arg(long, value_enum, default_value_t = GraphBackend::Falkor)]
    backend: GraphBackend,

    #[arg(long, default_value_t = 100)]
    batch_size: usize,
}
```

This is idiomatic Rust application code:

- `PathBuf` is an owned filesystem path.
- `usize` is the natural size type for counts and slice indexes.
- `InputFormat` and `GraphBackend` are enums, not strings.

Using enums matters. A stringly typed CLI lets invalid values drift deep into the program. A `ValueEnum` makes invalid values fail at parsing time.

```
#[derive(Debug, Clone, Copy, ValueEnum)]
enum InputFormat {
    Export,
    TalkRecords,
}
```

`Clone` and `Copy` are appropriate here because the enum is tiny and has no heap data. Passing it around by value is simpler than borrowing it.

3. Turning CLI Arguments Into Configuration

The CLI struct is specific to the binary. The library uses a separate config:

```rust
pub struct GraphLoadConfig {
    pub backend: GraphBackend,
    pub redis_url: String,
    pub helix_url: String,
    pub surreal_url: String,
    pub graph: String,
    pub replace: bool,
    pub batch_size: usize,
}
```

The conversion is implemented with `From`:

```rust
impl From<&Args> for GraphLoadConfig {
    fn from(args: &Args) -> Self {
        Self {
            backend: args.backend,
            redis_url: args.redis_url.clone(),
            helix_url: args.helix_url.clone(),
            surreal_url: args.surreal_url.clone(),
            graph: args.graph.clone(),
            replace: args.replace,
            batch_size: args.batch_size,
            // other fields omitted
        }
    }
}
```

This is one of the first places ownership matters.

`Args` owns its `String` fields. `GraphLoadConfig` also needs to own strings, because it may outlive the local `Args` borrow. Therefore, the conversion clones strings.

For small configuration strings, cloning is the right tradeoff. It avoids lifetime complexity and keeps the public config straightforward.

4. The Neutral Graph Model

The core data model is in `src/graph_loader.rs`:

```rust
#[derive(Debug, Clone, Default)]
pub struct GraphData {
    pub nodes: Vec<GraphNode>,
    pub edges: Vec<GraphEdge>,
}

#[derive(Debug, Clone)]
pub struct GraphNode {
    pub label: String,
    pub props: BTreeMap<String, GraphValue>,
}

#[derive(Debug, Clone)]
pub struct GraphEdge {
    pub from: String,
    pub to: String,
    pub relationship: String,
}
```

This is deliberately plain. A node has:

- one primary label
- a stable string ID in `props["id"]`
- a property map

An edge has:

- source node ID
- target node ID
- relationship name

The database-specific node IDs are not part of this model. That keeps the graph portable across backends.

Why `BTreeMap`?

Properties are stored in a `BTreeMap`:

```rust
pub props: BTreeMap<String, GraphValue>
```

A `HashMap` would also work. `BTreeMap` has one practical advantage here: deterministic iteration order. That makes generated Cypher, SurrealQL, and test output more stable.

Determinism is useful in loader code because query generation is easier to debug when the same input produces the same string order.

5. Modeling Property Values

The loader does not accept arbitrary JSON internally. It uses a small enum:

```
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum GraphValue {
    String(String),
    StringArray(Vec<String>),
}
```

This is a key design choice. The export data may start as JSON, but the loader supports only the value types it knows how to write safely across all backends.

That buys three things:

- each backend writer handles a finite set of cases
- unsupported values are converted at the importer boundary
- tests can compare values directly with `PartialEq`

The implementation also defines convenient conversions:

```
impl From<String> for GraphValue {
    fn from(value: String) -> Self {
        Self::String(value)
    }
}

impl From<&str> for GraphValue {
    fn from(value: &str) -> Self {
        Self::String(value.to_string())
    }
}
```

This lets calling code write:

```
("name", person.name.clone().into())
```

instead of:

```
("name", GraphValue::String(person.name.clone()))
```

6. Borrowing for Required Node IDs

Every node must have a string `id` property. The method that enforces this is:

```
impl GraphNode {
    fn id(&self) -> Result<&str> {
        match self.props.get("id") {
            Some(GraphValue::String(id)) => Ok(id),
            _ => bail!("all graph nodes must include a string id property"),
        }
    }
}
```

This is an excellent borrow-checker example.

The node owns the `String` inside `GraphValue::String`. The `id` method does not clone that string. It returns `&str`, a borrowed view into the owned string.

The returned reference cannot outlive `self`. Rust enforces that automatically. That is why this signature is safe:

```
fn id(&self) -> Result<&str>
```

The caller can use the borrowed ID while it is still working with the borrowed node, but cannot stash it somewhere longer-lived without copying it.

This keeps query generation cheap:

```
cypher_string(node.id()?)
```

No allocation is needed just to read the ID.

7. Error Handling With `anyhow`

The loader is an application-level tool, not a reusable low-level crate. It uses `anyhow::Result`:

```
use anyhow::{Context, Result, bail};
```

`bail!` returns an error immediately:

```
bail!("all graph nodes must include a string id property")
```

`Context` adds useful information to lower-level errors:

```
let graph_json = fs::read_to_string(input)
    .with_context(|| format!("failed to read {}", input.display()))?;
```

The `?` operator is central. It means:

- if the operation succeeds, unwrap the value
- if it fails, return the error from the current function

This keeps fallible code linear and readable.

8. Reading the Consolidated Export

The default importer reads one JSON file into typed structs:

```rust
fn read_export_graph(input: &PathBuf) -> Result<GraphData> {
    let graph_json = fs::read_to_string(input)
        .with_context(|| format!("failed to read {}", input.display()))?;
    let export: GraphExport = serde_json::from_str(&graph_json)
        .with_context(|| format!("failed to parse {}", input.display()))?;
    export.to_graph_data()
}
```

Notice the ownership path:

1. `fs::read_to_string` returns an owned `String`.
2. `serde_json::from_str` borrows that string while parsing.
3. `GraphExport` owns the parsed data.
4. `to_graph_data` borrows `GraphExport` and creates owned `GraphData`.

The export structs mirror the JSON:

```rust
#[derive(Debug, Deserialize)]
struct GraphExport {
    source_urls: Vec<String>,
    conferences: Vec<Conference>,
    meetups: Vec<MeetupGroup>,
    people: Vec<Person>,
    talks: Vec<Talk>,
    edges: Vec<Edge>,
}
```

These structs are private to the binary. That is a good boundary: the JSON format is an input concern, not a graph-loading concern.

9. Converting Export Records Into Nodes

Each export entity gets a small converter:

```
fn meetup_node(meetup: &MeetupGroup) -> GraphNode {
    let mut props = props([
        ("id", meetup.id.clone().into()),
        ("name", meetup.name.clone().into()),
        ("url", meetup.url.clone().into()),
    ]);
    insert_optional(&mut props, "timezone", meetup.timezone.as_deref());
    GraphNode::new("Meetup", props)
}
```

This function borrows a `MeetupGroup` and returns an owned `GraphNode`.

Why clone? Because the graph node must own its property strings. The source `MeetupGroup` is only borrowed. Moving fields out of a borrowed struct is not allowed, and should not be allowed: the caller still owns the export.

`as_deref` is a subtle useful method:

```
meetup.timezone.as_deref()
```

It turns `Option<String>` borrowed through `&MeetupGroup` into `Option<&str>`. That matches the helper:

```
fn insert_optional(props: &mut BTreeMap<String, GraphValue>, key: &str, value: Option<&str>)
```

This avoids cloning optional strings unless they are actually present and non-empty.

10. Building Maps With Const Generics

The helper for property maps is:

```
fn props<const N: usize>(items: [(&str, GraphValue); N]) -> BTreeMap<String, GraphValue> {
    items
        .into_iter()
        .map(|(key, value)| (key.to_string(), value))
        .collect()
}
```

`const N: usize` lets the function accept arrays of any length while preserving static type information.

These all compile:

```
props([("id", "person:1".into())])

props([
    ("id", "meetup:1".into()),
    ("name", "Graph Night".into()),
    ("url", "https://example.test".into()),
])
```

Without const generics, you would likely accept a slice or vector. The array version is compact at call sites and allocation-free before the final map is collected.

11. Optional Properties and Mutable Borrows

Optional fields are inserted with:

```
fn insert_optional(props: &mut BTreeMap<String, GraphValue>, key: &str, value: Option<&str>) {
    if let Some(value) = value.filter(|value| !value.is_empty()) {
        props.insert(key.to_string(), value.into());
    }
}
```

This function takes `&mut BTreeMap<...>`.

That mutable borrow means the caller temporarily gives the helper exclusive access to the map. While the helper is running, no other code can read or write the same map. After the function returns, the borrow ends and the caller can use the map again.

This is the borrow checker protecting you from iterator invalidation and concurrent mutation bugs.

12. Alternate Importer: Per-Talk Records

The alternate importer accepts Claude-style talk record files:

```
#[derive(Debug, Deserialize)]
struct TalkRecord {
    nodes: Vec<TalkRecordNode>,
    edges: Vec<TalkRecordEdge>,
}
```

It also accepts the enriched graph-record export produced by `scripts/enrich_meetup_entities.py`. That file uses the same top-level `nodes` and `edges` shape, but its node labels include `Speaker`, `Company`, and `Project` in addition to talks, meetups, and conferences.

It supports flexible field names:

```
#[derive(Debug, Deserialize)]
struct TalkRecordNode {
    id: String,
    #[serde(alias = "type", alias = "kind")]
    label: String,
    #[serde(default)]
    properties: serde_json::Map<String, serde_json::Value>,
}
```

`serde(alias = "...")` is useful when ingesting adjacent formats. It lets the Rust code keep one field name, `label`, while accepting JSON that says `type` or `kind`.

The importer deduplicates into maps:

```
let mut nodes_by_id: BTreeMap<String, GraphNode> = BTreeMap::new();
let mut edges_by_key: BTreeMap<(String, String, String), GraphEdge> = BTreeMap::new();
```

Node identity is the stable string ID. Edge identity is the tuple:

```
(from, to, relationship)
```

This mirrors how graph loaders usually think about idempotence.

13. Moving Values Into Deduplication Maps

The edge deduplication code is:

```
edges_by_key
    .entry((
        edge.from.clone(),
        edge.to.clone(),
        edge.relationship.clone(),
    ))
    .or_insert_with(|| GraphEdge {
        from: edge.from,
        to: edge.to,
        relationship: edge.relationship,
    });
```

This is a compact ownership lesson.

The map key needs owned strings, so the key clones the fields. If the edge is not already present, `or_insert_with` moves the original strings into the stored `GraphEdge`.

Why not avoid the clones? You could restructure this code to clone less, but this version is simple and correct. For a loader dominated by database I/O, the small string clones are not the bottleneck.

Rust makes the cost visible, which lets you choose intentionally.

14. Converting JSON Values Safely

The alternate importer converts arbitrary JSON into the loader's smaller property enum:

```
fn json_graph_value(value: serde_json::Value) -> GraphValue {
    match value {
        serde_json::Value::String(value) => GraphValue::String(value),
        serde_json::Value::Array(values) => GraphValue::StringArray(
            values
                .into_iter()
                .filter_map(|value| match value {
                    serde_json::Value::String(value) if !value.is_empty() => Some(value),
                    _ => None,
                })
                .collect(),
        ),
        serde_json::Value::Null => GraphValue::String(String::new()),
        other => GraphValue::String(other.to_string()),
    }
}
```

This is boundary normalization. Inside the loader, backends do not need to deal with arbitrary JSON. They only handle `String` and `StringArray`.

The function takes `serde_json::Value` by value, not by reference. That means it can move strings out of the JSON without cloning:

```
serde_json::Value::String(value) => GraphValue::String(value)
```

If it took `&serde_json::Value`, this branch would need to clone the string.

15. Public API, Private Implementation

The public function stays small:

```
pub fn load_graph(config: &GraphLoadConfig, graph: &GraphData) -> Result<()> {
    match config.backend {
        GraphBackend::Falkor => FalkorLoader.load(config, graph),
        GraphBackend::HelixHttp => HelixHttpLoader.load(config, graph),
        GraphBackend::HelixRustSdk => HelixRustSdkLoader.load(config, graph),
        GraphBackend::Surrealdb => SurrealHttpLoader.load(config, graph),
        GraphBackend::SurrealdbRustSdk => SurrealRustSdkLoader.load(config, graph),
    }
}
```

The internal abstraction is a trait:

```
trait GraphLoader {
    fn load(&self, config: &GraphLoadConfig, graph: &GraphData) -> Result<()>;
}
```

The trait is private. That is deliberate.

The outside world does not need to know how backend dispatch works. It only needs `load_graph`. Keeping the trait private gives us freedom to change the internal design without breaking callers.

16. Why Not Return Trait Objects?

Another design would be:

```
fn backend_loader(config: &GraphLoadConfig) -> Box<dyn GraphLoader>
```

That is useful when the selected backend must be stored and reused as a runtime object. This loader does not need that. It dispatches once, performs the load, and exits.

The current match is simpler:

- no heap allocation for a boxed trait object
- no dynamic dispatch beyond the simple call
- easy to read
- easy to debug

Rust rewards choosing the simplest abstraction that fits the lifetime of the problem.

17. Backend-Specific Ownership

Each backend borrows the neutral graph and generates backend-specific commands.

For FalkorDB:

```
for node in &graph.nodes {
    let query = format!(
        "MERGE (n:{} {{id:{}}}) SET n += {}",
        falkor_labels(node),
        cypher_string(node.id()?),
        cypher_map(&node.props)
    );
    falkor_query(&mut connection, &config.graph, &query)?;
}
```

Important details:

- `&graph.nodes` iterates by shared reference.
- `node.id()?` borrows the ID.
- `cypher_map(&node.props)` borrows the property map.
- `query` is an owned string created for the database call.
- `connection` is borrowed mutably because Redis commands mutate connection state.

That last point is a common Rust pattern. Network clients and database connections often require `&mut self` because sending a command changes buffers, sequence state, or protocol state.

18. Escaping Strings for Cypher

Cypher strings are generated explicitly:

```rust
fn cypher_string(value: &str) -> String {
    let mut escaped = String::with_capacity(value.len() + 2);
    escaped.push('\'');
    for ch in value.chars() {
        match ch {
            '\\' => escaped.push_str("\\\\"),
            '\'' => escaped.push_str("\\'"),
            '\n' => escaped.push_str("\\n"),
            '\r' => escaped.push_str("\\r"),
            '\t' => escaped.push_str("\\t"),
            _ => escaped.push(ch),
        }
    }
    escaped.push('\'');
    escaped
}
```

The function borrows `&str` and returns a new escaped `String`.

`String::with_capacity(value.len() + 2)` is a small performance hint. The output will be at least two bytes longer because of the surrounding quotes. Escapes may make it longer, but this capacity avoids reallocating for the common case.

19. Backend Differences Shape the Code

The same graph model is written differently by each database.

FalkorDB is Cypher-like:

```
MERGE (n:Talk {id:'talk:1'}) SET n += {title:'Graph Loading'}
```

HelixDB uses dynamic graph mutations:

```
g().add_n(node.label.clone(), helix_sdk_properties(node))
```

SurrealDB uses records and edge tables:

```
CREATE type::record("talk", "talk:1") SET title = "Graph Loading";
RELATE (type::record("person", "person:1"))->presents->(type::record("talk", "talk:1"));
```

The abstraction works because all three can honor the same portable contract:

- stable string node IDs
- labels or tables derived from node labels
- relationships derived from edge names

20. Where Helix Affects the Property Model

Live Helix rejected array-valued properties in graph mutations. That directly affects the backend writer:

```rust
fn helix_http_properties(node: &GraphNode) -> Vec<serde_json::Value> {
    node.props
        .iter()
        .filter_map(|(key, value)| match (key.as_str(), value) {
            ("labels", _) => None,
            (_, GraphValue::String(value)) => Some(json!([key, {"Value": {"String": value}}])),
            (_, GraphValue::StringArray(_)) => None,
        })
        .collect()
}
```

This code says:

- never send the synthetic `labels` property
- send string properties
- omit string arrays

That is backend-specific behavior hidden behind the loader. FalkorDB and SurrealDB can preserve arrays; Helix currently cannot through this mutation path.

The portable export keeps string alternatives such as `tags_json` and `tags_csv` for this reason.

21. Batching With Slices

Backends use batch size like this:

```
for chunk in graph.nodes.chunks(config.batch_size.max(1)) {
    post_helix(&client, &config.helix_url, &helix_add_nodes_request(chunk)?)?;
}
```

`chunks` returns borrowed slices:

```
&[GraphNode]
```

No nodes are copied. The batch function only sees a borrowed window over the existing vector.

`config.batch_size.max(1)` prevents a panic. A chunk size of zero is invalid, so the loader clamps it to at least one.

22. Sync and Async Together

Some SDKs are async. The public loader function is synchronous:

```
pub fn load_graph(config: &GraphLoadConfig, graph: &GraphData) -> Result<()>
```

The SurrealDB SDK backend bridges the two with a Tokio runtime:

```
impl GraphLoader for SurrealRustSdkLoader {
    fn load(&self, config: &GraphLoadConfig, graph: &GraphData) -> Result<()> {
        let runtime = tokio::runtime::Runtime::new()
            .context("failed to create Tokio runtime")?;
        runtime.block_on(load_surrealdb_rust_sdk_async(config, graph))
    }
}
```

This is pragmatic for a command-line loader. The binary can stay simple while individual backends use async where required.

For a long-running server, you would probably make the public API async instead of creating runtimes internally.

23. Lifetimes Without Writing Lifetimes

Most of this project does not write explicit lifetime parameters. Rust infers them.

For example:

```
fn insert_optional(props: &mut BTreeMap<String, GraphValue>, key: &str, value: Option<&str>)
```

The references only need to live for the duration of the function call.

Another example:

```
fn id(&self) -> Result<&str>
```

Rust understands that the returned `&str` is tied to `&self`.

You usually write explicit lifetimes only when a function returns a borrowed value and there is more than one possible input lifetime. This code mostly has obvious borrowing relationships, so elision keeps it readable.

24. Tests as Design Locks

The loader has tests for behavior that should not regress:

```rust
#[test]
fn helix_http_node_batch_omits_arrays() {
    let graph = sample_graph();
    let request = helix_add_nodes_request(&graph.nodes).unwrap();
    let properties = &request["query"]["queries"][0]["Query"]["steps"][0]["AddN"]["properties"];
    assert!(
        !properties
            .as_array()
            .unwrap()
            .iter()
            .any(|prop| prop[0] == "tags")
    );
}
```

This test is not just checking syntax. It captures a live backend constraint: Helix should not receive array properties.

Another test locks the alternate importer behavior:

```rust
#[test]
fn converts_talk_records_to_backend_neutral_graph() {
    // Builds a TalkRecord directly and checks the resulting GraphData.
}
```

Good tests describe intent. They make future refactoring safer.

25. Practical Borrow Checker Rules From This Project

Here are the borrow checker lessons this loader teaches.

Prefer borrowing large inputs:

```
pub fn load_graph(config: &GraphLoadConfig, graph: &GraphData) -> Result<()>
```

Return owned output when transforming formats:

```
fn read_export_graph(input: &PathBuf) -> Result<GraphData>
```

Clone at boundaries when a new owner is needed:

```
("name", person.name.clone().into())
```

Take mutable references for helper functions that update a structure:

```
insert_optional(&mut props, "timezone", meetup.timezone.as_deref());
```

Move values when consuming parsed input:

```
fn json_graph_value(value: serde_json::Value) -> GraphValue
```

Use slices for batches:

```
fn helix_add_nodes_request(nodes: &[GraphNode]) -> Result<serde_json::Value>
```

26. How the Abstractions Were Chosen

The main abstraction is the neutral graph:

```
GraphData
GraphNode
GraphEdge
GraphValue
```

That abstraction exists because the scraper should not know how FalkorDB, HelixDB, or SurrealDB write graph data.

The second abstraction is backend selection:

```
GraphBackend
GraphLoadConfig
load_graph(...)
```

That abstraction exists because the CLI needs to select a backend at runtime.

The third abstraction is internal:

```
trait GraphLoader
```

That abstraction exists because backend implementations should have a common shape while keeping the public API stable.

A useful rule: abstractions should protect a real boundary. Here the real boundaries are:

- input format versus graph model
- graph model versus database writes
- public API versus private backend implementation

27. What Could Improve Next

The current implementation is intentionally practical. Good next steps would be:

- add automated live integration tests for all three databases
- add backend-specific read helpers for common graph traversals
- support typed numeric and date properties in `GraphValue`
- add backend upsert modes for Helix and SurrealDB where available
- split backend modules into separate files once the single module becomes too large

Do not split files just to split files. Split when navigation, ownership, or testability improves.

Conclusion

This graph loader is a useful Rust learning project because it is small enough to understand and real enough to show the language's strengths.

It demonstrates:

- typed command-line parsing
- owned data models
- borrowed public APIs
- fallible conversion with `Result`
- structured backend dispatch
- careful string generation
- importer normalization
- tests that encode backend constraints

The most important Rust idea here is ownership at boundaries. Parse into owned data. Borrow while loading. Clone only when a second owner is actually needed. Move values when consuming an input format. Keep backend-specific details behind a stable interface.

That is the heart of writing clear Rust.